

Lecturer

Prof. Dr. Christian Tschudin (christian.tschudin@unibas.ch)

Teaching Assistants

Sebastian Lenzlinger (sebastian.lenzlinger@unibas.ch)

Mark Starzynski (mark.starzynski@unibas.ch)

Assistant

Ali Ajorian (ali.ajorian@unibas.ch)

Uploaded on

Thu., 14. March 2024

Deadline

Sun., 24. March 2024 (23:55)

Upload to ADAM

Modalities

The exercises have to be done in **groups of 2 students**. Upload only the final version of the exercise to Adam and specify your group partner.

Please upload your short report, containing all answers to the asked questions as a .pdf file to ADAM by the deadline at the latest.

Wireshark

In this exercise sheet you will use Wireshark to record and analyse network traffic. Download and install Wireshark at <https://www.wireshark.org>. (Note for Windows users: Make sure you also install Npcap, which comes with the Wireshark installer to scan packets on the loopback device (localhost)).

Task 1 - TCP Traffic (3 points)

- (a) You are supplied a program `tcp_chat.py`. First, start up Wireshark and have it listen on the `localhost` interface (127.0.0.1). Next, in one terminal start up the server on `localhost` and use `telnet` in a different terminal to connect to it. When you type text into one terminal and hit enter, you should be able to see that message in the other terminal.

Your task is to record the traffic generated between the server and the client (telnet in this case) and then report on the involved protocols and the contents of the packets when invoking `tcp_chat.py`.

Using screenshots, report on the packets and their respective contents: 1. When the connection is first established. 2. When messages are exchanged. 3. When the connection is terminated. Explain the contents of the relevant packets and how they relate to relevant protocols.

- (b) Now write a client program to interact with the provided server. Fill in the empty `start_client()` method in `tcp_chat.py`.

Task 2 - UDP Chat (3 points)

Implement an UDP chat application, where a central server mediates direct connections between clients. The application must be fully operable from the command line.

To establish a chat between two clients, they both first have to log in to the rendez-vous server using a unique nickname. The server maintains a list containing a mapping between nicknames and the corresponding IP addresses and port numbers. One of the clients then sends the nickname of the desired communication partner, and the server returns the matching IP address and port number. Now both clients are able to exchange text messages directly, without a connection (no text messages are sent via the server).

Task 3 - Broadcast UDP Chat (3 points)

Now implement a broadcast variant of the UDP chat. In this version, clients do not rely on a server for log-in anymore, instead the clients broadcast their messages to a broadcast address, and all clients listen to messages coming from that address. The application must be fully operable from the command line.

As in the task before, clients should be able to choose a nickname. This nickname needs to be displayed to the other clients upon receiving a message, so you can implement a small communication protocol to accomplish this.

Question: Suppose you want to extend your chat application to show the user a current list of all the other participants in the network. How would you implement this, without a server still? How would you deal with joining/leaving clients? At which layer of the network model ("Internet protocol stack") do you solve this problem? Are there any downsides to this approach?

Task 4 - HTTPS Traffic (1 point)

Start recording network traffic with Wireshark and open any `https` website (e.g. `https://zenofpython.org/`). What is a big difference in the content of the relevant packets, especially with respect to Task 1?